



Сформируй чёткую 6D-формулу “белой росы” и затем переписать в Python-класс как отдельный режим внутри общей системы

Да — это можно оформить строго: в режиме «белой росы» 6-я координата становится **параметром порядка**, который описывает переход системы из “сырого” возбуждённого состояния в **упорядоченный, когерентный режим**, а не в другой материал ^{[1] [2]}. Такой ход хорошо согласуется с динамикой параметров порядка и self-organized transitions, где медленная переменная управляет сменой коллективного режима, а не просто накапливает амплитуду ^{[3] [4]}.

6D-формула

Зафиксируем состояние как

$$\mathbf{s}(t) = (x, y, z, h, e, \rho),$$

где x, y, z — внутренняя 3D-геометрия/динамика, h — активность, e — ресурс, а ρ — **параметр порядка “белой росы”** ^{[1] [2]}.

Тогда систему можно задать так:

$$\dot{x} = y, \quad \dot{y} = z, \quad \dot{z} = -a_x x - a_y y + b_h \tanh(h) + b_\rho \tanh(\rho),$$

где $[1 - \rho^2]_+ = \max(0, 1 - \rho^2)$ ^{[3] [1]}. Здесь ρ не является ни “топливом”, ни “модератором”: это именно **медленная переменная фазовой самоорганизации**, которая сначала растёт при достаточном ресурсе, а затем сама ограничивает грубую активность и переводит систему в более упорядоченный режим ^{[2] [4]}.

Смысл членов

Ключевой член $-\nu\rho^3$ делает рост ρ **самоограниченным**, как у типичного параметра порядка около бифуркации, а множитель $[1 - \rho^2]_+$ в уравнениях активности и ресурса означает, что по мере формирования “росы” возбуждение уже не может бесконечно усиливаться ^{[3] [5]}. Это даёт именно то, что вы описали: уран не исчезает и не превращается в плутоний или тяжёлую воду, а **входит в иное качественное состояние организации** ^{[1] [2]}.

Python-класс

Как отдельный режим внутри общей системы, это удобнее всего оформить как подкласс, который переопределяет только закон эволюции 6-й координаты и эффективный драйв:

```
from dataclasses import dataclass
import math

@dataclass
class WhiteDewConfig:
    dt: float = 0.01
    beta: float = 6.0
    theta: float = 0.25
    g: float = 1.1
    saturation: float = 1.0
    ax: float = 0.6
    ay: float = 0.25
    bh: float = 0.15
    brho: float = 0.10
    gamma_h: float = 0.08
    h_star: float = 0.0
    p: float = 0.18
    mu: float = 0.05
    r: float = 0.08
    eps: float = 0.08
    alpha: float = 1.2
    lam: float = 0.45
    nu: float = 0.7
    kappa: float = 0.35
    xi: float = 0.25

class WhiteDewMode:
    def __init__(self, cfg: WhiteDewConfig | None = None):
        self.cfg = cfg or WhiteDewConfig()

    @staticmethod
    def sigmoid(x: float) -> float:
        return 1.0 / (1.0 + math.exp(-x))

    @staticmethod
    def sat(x: float, limit: float) -> float:
        return limit * math.tanh(x / max(limit, 1e-9))

    def derivatives(self, s: dict, t: float) -> dict:
        x, y, z = s["x"], s["y"], s["z"]
        h, e, rho = s["h"], s["e"], s["rho"]

        gate = self.sigmoid(self.cfg.beta * (e - self.cfg.theta))
        order_brake = max(0.0, 1.0 - rho * rho)
        drive = self.sat(self.cfg.g * gate * order_brake, self.cfg.saturation)
        ext = 0.55 * math.sin(1.7 * t + x) + 0.35 * math.cos(2.3 * t + y)

        dx = y
        dy = z
        dz = -self.cfg.ax * x - self.cfg.ay * y + self.cfg.bh * math.tanh(h) + self.cfg
```

```

dh = ext + drive - self.cfg.gamma_h * (h - self.cfg.h_star)
de = self.cfg.p - self.cfg.mu * e - drive + self.cfg.r * max(0.0, self.cfg.h

drho = self.cfg.eps * (
    self.cfg.alpha * gate
    - self.cfg.lam * rho
    - self.cfg.nu * rho**3
    + self.cfg.kappa * math.tanh(h)
    - self.cfg.xi * abs(h - self.cfg.h_star) * rho
)

return {"x": dx, "y": dy, "z": dz, "h": dh, "e": de, "rho": drho, "gate": g

```

Этот класс реализует именно режим **White Dew**, где ρ — параметр порядка, а не ещё одна “энергетическая” координата [1] [2].

В общей системе

Если в вашей общей архитектуре уже есть режимы **ActivatorModerator6D** и, например, “критический” режим, то **WhiteDewMode** лучше включать как **альтернативную динамику шестой координаты и эффективного драйва**, а не как полностью отдельную систему. То есть общая база остаётся одной и той же, а “белая роса” становится **режимом фазовой самоорганизации**, который можно включать при выполнении условия $e > \theta$ и умеренном отклонении активности от целевого уровня [6] [7].

Следующим шагом я могу собрать это уже в **один чистый Python-файл с базовым классом и двумя режимами: ActivatorModerator6D и WhiteDewMode**, чтобы переключение происходило параметром `mode="white_dew"`.



1. <https://cris.biu.ac.il/en/publications/order-parameter-dynamics-in-complex-systems-from-models-to-data>
2. <https://pubs.aip.org/aip/cha/article/34/2/022101/3261859/Order-parameter-dynamics-in-complex-systems-From>
3. <https://link.aps.org/doi/10.1103/PhysRevLett.116.240601>
4. <https://pubmed.ncbi.nlm.nih.gov/18026746/>
5. <https://link.aps.org/doi/10.1103/PhysRevE.76.061110>
6. <https://openreview.net/forum?id=d16mJDyQN6>
7. <https://www.sciencedirect.com/science/article/pii/S0168010226000088>
8. <https://arxiv.org/abs/2602.01259>
9. <https://pubmed.ncbi.nlm.nih.gov/12513576/>
10. <https://arxiv.org/html/2110.13524v3>